

Fondements informatiques I : Initiation au Python

TD 1

1 Révision Python

Exercice 1.1 *Qu'affiche ce programme ?*

```
def mul2(a):  
    return a * 2  
  
def div2(a):  
    return a // 2  
  
a = 8  
  
v1 = mul2(a)  
v2 = div2(a)  
print(a, v1, v2)  
  
a = mul2(a)  
a = div2(a)  
  
print(a)
```

Exercice 1.2 *Que font les programmes suivants ?*

```
1. for i in range(5):  
    j = i + 1  
    i+=j  
  
    print("i et j valent: ", i, j)  
2. for i in range(5):  
    for j in range(5):  
        for k in range(5):  
            print("i+j+k")  
3. a = 100  
    b = 1000  
  
    def fun1(a):  
        a+=1  
        return a  
  
    def fun2(a):  
        b=3  
        a += b
```

```

    return a

def fun3(a):
    global b
    a += b
    return a

fun1(a)
print("a =", a)
a = fun2(a)
print("a et b valent", a, "et", b)
a = fun3(a)
print("a =", a)
4. def rajouter_item(lst):
    lst.append(10)

a = [1, 2, 3]
rajouter_item(a)
print(a)

```

2 Algorithmique et programmation en Python

Exercice 2.1

Nous fixons l'ordre $a < b < c \dots x < y < z$ sur les lettres de l'alphabet. Pour deux chaînes de caractères $A = a_1a_2 \dots a_n$ et $B = b_1b_2 \dots b_m$, on dit que $A < B$ si et seulement si :

- soit $a_1 < b_1$
- soit il existe un indice k , $1 \leq k < n$, tel que $a_1 = b_1, \dots, a_k = b_k$ et $a_{k+1} < b_{k+1}$
- soit $a_1 = b_1, \dots, a_n = b_n$ et $n < m$.

Cette relation d'ordre s'appelle ordre lexicographique et nous souhaitons ordonner les mots d'un tableau suivant l'ordre lexicographique comme dans un dictionnaire de la langue française.

1. Nous souhaitons comparer deux chaînes de caractères $a_1a_2 \dots a_n$ et $b_1b_2 \dots b_m$. Donner le pseudocode de l'algorithme.
2. Nous souhaitons trier en ordre lexicographique les mots dans un tableau. En vous inspirant sur les algorithmes vues en cours, et sur la procédure précédente, proposez un algorithme de tri du tableau. Vous détaillerez toutes les étapes du fonctionnement de votre algorithme sur le tableau de mots suivants :

```
tal=["etoile", "chocolat", "chaussette", "cadeau", "sapin"]
```

3. Écrire le code en Python de cet algorithme (et le tester à la maison). Comment fait-on pour comparer deux chaînes de caractères en Python ?

Exercice 2.2 Dans certaines séquences d'ADN, on observe des motifs dits **palindromiques** : un fragment est suivi de sa version complémentaire inversée. On souhaite modéliser ce phénomène à l'aide de chaînes de caractères représentant des bases d'ADN. On suppose que l'ADN est composé de quatre bases :

A, T, C, G

et que les bases complémentaires sont :

$A \leftrightarrow T, \quad C \leftrightarrow G.$

1. Donner une procédure **complement** qui prend en entrée une chaîne d'ADN et renvoie la séquence complémentaire de cette chaîne.

Exemple :

"ATCG" → "TAGC"

2. On définit une séquence miroir comme une séquence formée en ajoutant à une séquence donnée son complément inversé.

Exemple :

$$\begin{aligned} S_0 &= "A", & S_1 &= "A" + \text{complément inversé de "A"} = "A" + "T" = "AT", \\ S_2 &= "AT" + \text{complément inversé de "AT"} = "AT" + "AT" = "ATAT" \end{aligned}$$

Donner le pseudocode d'un algorithme qui à partir d'une séquence de départ S_0 construit la séquence miroir obtenue après n étapes. Pour cela, vous utiliserez la procédure de la question 1.

3. Donner un algorithme permettant de vérifier qu'une chaîne de caractères donnée est bien une séquence miroir. Si cela est vrai, votre algorithme devra renvoyer le nombre d'étapes n effectuées pour la séquence.
4. Écrire le code Python pour les algorithmes des questions 1,2,3 (et testez-le chez vous).